MARINE AUTOMATION PORTFOLIO · PROJECT 03

AUTONOMOUS MARINE POWER MANAGEMENT SYSTEM (PMS)

Intelligent Load-Shedding, Auto-Synchronising and Blackout Recovery

TECHNICAL PROJECT MANUAL · REV. A

By Sipho Lucky Sibanda

PLC Platform: Siemens S7-1500 (SCL) / CODESYS-portable Structured Text

Simulation & Portfolio Engineering Build · Not for Shipboard Deployment

0i

FRONT MATTER

Document Control & Disclaimer

This document is a self-authored technical project manual produced as part of a personal engineering portfolio. It describes the design, control philosophy, and simulated validation of a marine Power Management System (PMS) covering fast load shedding, auto-synchronising, and blackout recovery, built to demonstrate electrical sequencing logic, state-machine design, and marine regulatory awareness for automation, controls, and marine electrical engineering roles.

The system described here was developed and tested in simulation only (PLCSIM-style forcing of inputs and desktop review), using publicly available regulatory references for realism. No part of this project has been installed, commissioned, or verified on physical shipboard hardware, and it must not be treated as a certified or classed Power Management System.

SAFETY CRITICAL — Portfolio project, not a certified PMS

A real shipboard PMS requires generator and governor/AVR characterisation specific to the actual machinery installed, full load-sharing (droop) control, protection coordination studies, and class society approval before any of this logic could be considered for actual use. Figures, thresholds, and I/O in this manual are engineering-realistic but illustrative.

Document Title	Autonomous Marine PMS — Technical Manual
Author	Sipho Lucky Sibanda
Revision	A
Document Type	Portfolio Technical Manual (Simulation)
Target PLC Platform	Siemens S7-1500 (TIA Portal / SCL) — CODESYS-portable
Related Repository	marine-pms-loadshare
Series	Marine Automation Portfolio — Project 03

ii

FRONT MATTER

Contents

iii

FRONT MATTER

Executive Summary

A ship that loses all electrical power isn't just inconvenienced — it's adrift, unable to steer, and at risk of collision or grounding until power is restored. This project simulates the PLC-side logic of a marine Power Management System, the software layer whose entire job is to make sure that scenario either never happens, or resolves itself in a matter of seconds rather than minutes.

Three distinct problems are covered, and the manual is deliberately structured around keeping them separate, because conflating them is where real PMS bugs come from. Fast load shedding reacts to a generator suddenly tripping, shedding pre-defined load blocks in the time it takes to open a contactor — there's no time for a clever calculation. Auto-synchronising is the opposite kind of problem: patient, state-machine-driven, bringing a second generator's speed, voltage, and phase into alignment with a live bus before ever closing a breaker onto it. Blackout recovery is a third problem again — starting from nothing, with a bus that's genuinely dead rather than live and needing synchronisation.

The function block that ties these together, FB_PMS_PowerManagement, deliberately keeps a dead-bus reclose (no phase matching needed, because there's nothing to phase-match against) as a completely separate code path from a live synchronising sequence — closing a breaker onto a live bus without verifying phase alignment first is precisely the kind of mistake that damages generators, and the two paths are structured so that mistake isn't possible to make by accident.

What follows documents the regulatory basis, architecture, hardware assumptions, full I/O list, control philosophy, complete annotated code, HMI design (including a working synchroscope), alarm philosophy, and the functional test procedure used to validate the logic in simulation.

01

PMS POWER MANAGEMENT

Regulatory & Marine Context

SOLAS Chapter II-1 is the backbone regulation behind why this project exists at all. Regulations 42 and 43 require passenger and cargo ships respectively to carry a self-contained emergency source of electrical power, independent of the main generating plant, capable of supplying defined essential services — steering gear, navigation lights, emergency lighting, and fire-fighting equipment among them — within a strict time limit after a blackout. For most vessel types, that limit is 45 seconds.

That single number — 45 seconds — is why blackout recovery in this project is automated rather than left to a duty engineer to run manually from a switchboard. A human being locating the right breaker, confirming conditions, and closing it by hand inside 45 seconds of a blackout, at night, possibly in heavy weather, is not a plan a class society will accept as the primary defence.

Beyond the emergency generator: getting the main plant back

SOLAS's 45-second requirement covers the emergency generator specifically. It says nothing about how quickly the main generating plant needs to be back online and carrying propulsion and normal ship's load — that's where classification society PMS notations and guidance (e.g. DNV's power management system requirements) come in, expecting a documented, largely automatic sequence for black-starting the main plant once the emergency source has stabilised the situation.

> **NOTE — Why load shedding gets its own regulatory attention**
> IACS and class society rules also expect a generating plant to survive the loss of its largest single generator without a full blackout, provided load shedding acts fast enough. That single requirement is effectively the specification for this project's fast load-shedding logic (Chapter 5) — it isn't a nice-to-have feature, it's the mechanism that's supposed to prevent a single generator trip from cascading into the exact blackout scenario Chapter II-1 exists to recover from.

Standards referenced

- SOLAS Chapter II-1, Regulations 42/43 — emergency source of electrical power and its distribution.
- IEC 61892-1 / IEC 60092 series — electrical installations on ships and offshore units.
- Class society PMS guidance — e.g. DNV-RU-SHIP Pt.4 Ch.8 on power management systems, covering load-dependent start/stop, load sharing, and blackout recovery expectations.

02

PMS POWER MANAGEMENT

System Architecture & Process Overview

Two main generators feed a common busbar through breakers, supervised by a synchronising relay that reports frequency, voltage, and phase angle whenever a paralleling attempt is in progress. An emergency generator sits on its own independent branch, exactly as SOLAS requires. Essential loads are wired to a section of the switchboard that no load-shed output can ever touch.

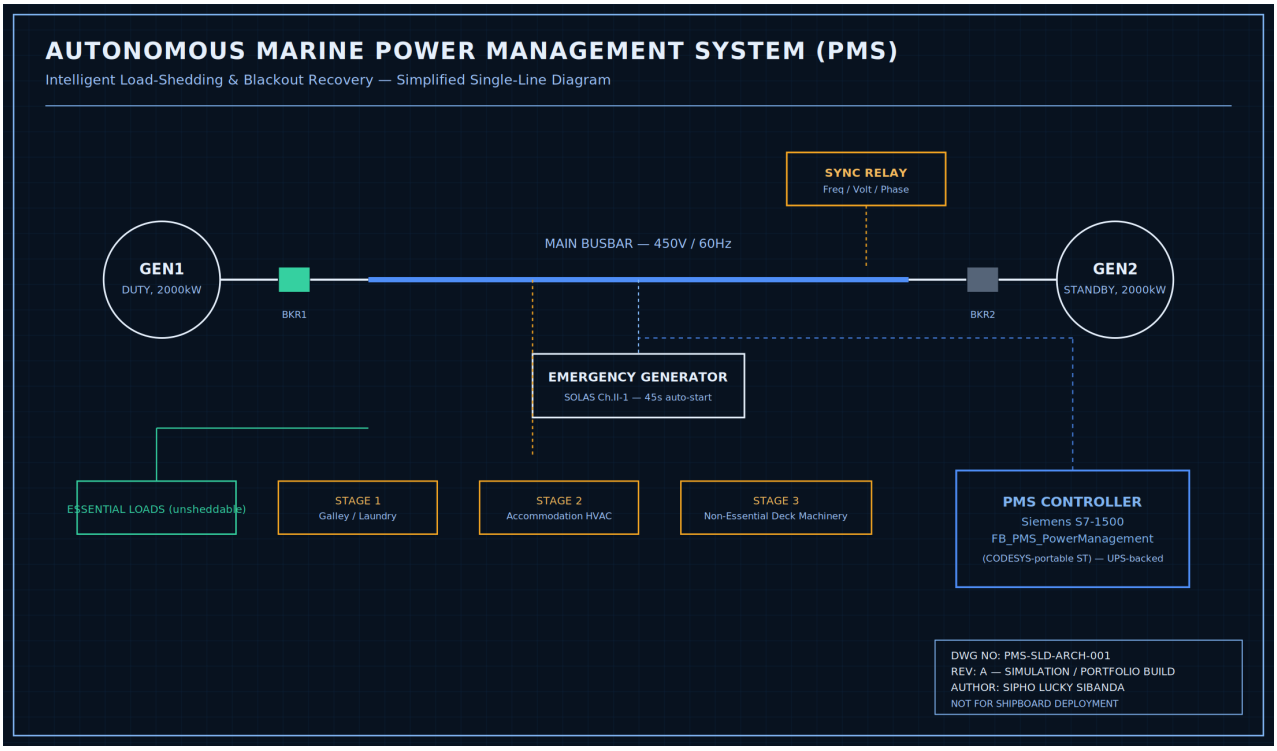

Figure 2.1 — Simplified single-line diagram. Solid lines represent power distribution; dashed lines represent PMS controller signal paths.

Control hierarchy

- Field layer — generator power meters (kW), synchronising relay (frequency/voltage/phase), breaker auxiliary contacts, governor and AVR bias inputs.
- Control layer — a single PLC function block, FB_PMS_PowerManagement, hosted on a Siemens S7-1500 (or CODESYS-based marine PMS controller), running load monitoring, the sync state machine, and blackout recovery.
- Supervisory / HMI layer — a switchboard console giving the watch-keeping engineer a single-line diagram view, a live synchroscope, and load-vs-capacity trending.

Why the PMS controller has to survive the blackout it's recovering from

It's worth stating explicitly: the PLC running this logic has to be on UPS or battery-backed power, independent of the main generating plant it's managing. A controller that itself goes dead in a blackout can't run a blackout recovery sequence. This is a hardware/power-architecture requirement that sits outside the Structured Text itself, but it's a precondition for any of Chapter 6's blackout logic to mean anything.

PMS POWER MANAGEMENT

03 Hardware & Instrumentation Specification

As with the other two projects in this series, this logic is platform-portable but written against a concrete reference platform so thresholds and timing are grounded.

Component	Representative Spec	Role
PMS Controller CPU	Siemens SIMATIC S7-1500, UPS/battery-backed supply	Executes FB_PMS_PowerManagement
Generator Power Meter	CT/PT-derived, 4–20mA or digital bus, per generator	Active power (kW) feedback
Synchronising Relay	Freq/volt/phase, dedicated sync relay per incoming breaker	Live-sync supervision
Governor Interface	4–20mA speed bias, -100 to +100%	Speed trim during synchronising
AVR Interface	4–20mA voltage bias, -100 to +100%	Voltage trim during synchronising
Generator Breaker	Motorised, 24VDC close/trip, auxiliary contact feedback	Isolation and paralleling
Emergency Generator	Self-contained start (battery/air), independent fuel supply	SOLAS emergency source
Load Contactors	24VDC, one per load-shed stage group	Non-essential load isolation

DESIGN NOTE — Why governor/AVR bias, not a direct setpoint

The synchronising logic in this project trims the incoming generator's governor and AVR with a bias signal layered on top of the generator's own local control, rather than overriding it outright. This mirrors real practice: the generator's own governor and AVR always retain their own protective limits, and the PMS is a supervisory trim on top, not a replacement for them.

04

PMS POWER MANAGEMENT

I/O List

The table below is the working I/O list for FB_PMS_PowerManagement (see also docs/IO_List.md in the repository, and Appendix A of this manual).

Inputs

Tag	Description	Signal	Range / Units
AI_Gen1_Load_kW	Gen1 active power	4–20mA	0–2200 kW
AI_Gen2_Load_kW	Gen2 active power	4–20mA	0–2200 kW
AI_Gen2_Freq_Hz	Gen2 (incoming set) frequency	Sync relay	45–65 Hz
AI_BusFreq_Hz	Main busbar frequency	Freq. relay	45–65 Hz
AI_Gen2_Voltage_V	Gen2 terminal voltage	Sync relay	0–500 V
AI_BusVoltage_V	Main busbar voltage	PT / volt. relay	0–500 V
AI_PhaseAngle_Deg	Gen2 vs Bus phase angle	Sync relay	-180 to +180°
DI_Gen1_Running / DI_Gen2_Running	Engine running feedback	24VDC digital	0/1
DI_Gen1_Breaker_Closed / DI_Gen2_Breaker_Closed	Breaker aux. contacts	24VDC digital	0/1
DI_EmergencyGen_Running	Emergency generator feedback	24VDC digital	0/1
DI_ManualSyncRequest	Operator sync request pushbutton	24VDC digital	0/1
DI_System_Enable	Master enable	24VDC digital	0/1

Outputs

Tag	Description	Signal
DO_Gen1_Start / DO_Gen2_Start	Generator start commands	24VDC digital
DO_Gen1_Breaker_Close	Dead-bus reclose (blackout recovery only)	24VDC digital
DO_Gen2_Breaker_Close	Live-sync breaker close (pulsed)	24VDC digital
AO_Gen2_GovernorBias_Pct / AO_Gen2_AVRBias_Pct	Speed/voltage trim during sync	4–20mA, ±100%
DO_LoadShed_Stage1/2/3	Non-essential load group trips	24VDC digital ×3

Tag	Description	Signal
DO_EmergencyGen_Start	Emergency generator start command	24VDC digital
Alarm_Overload / Alarm_Blackout / Alarm_SyncTimeout	Alarms	24VDC digital ×3
PMS_State / SystemStatus	State machine state / status text	Internal / HMI tag

PMS POWER MANAGEMENT

05 Control Philosophy: Three Linked Problems

This project's logic makes more sense once the three problems it solves are seen as genuinely different in character, not three variations on the same theme.

5.1 Fast load shedding: no time to calculate

When Gen1 trips unexpectedly while carrying most of the plant's load, the remaining capacity drops instantly, but the load doesn't — whatever's still running is still drawing the same power. If nothing intervenes within a very short window (the manufacturer's overload trip curve, typically low single-digit seconds at high overload), the surviving generator's own protection trips it too, and the plant blacks out anyway. There's no time to compute an optimal shedding amount; the logic sheds pre-sized load blocks (Chapter 3's load contactor groups) in a fixed, predictable order, decided in advance rather than in the moment.

5.2 Auto-synchronising: a patient, verified state machine

Bringing a second generator onto a live bus is the opposite problem: there's no emergency, and rushing it is actively dangerous. Closing a breaker while the incoming generator's phase is misaligned with the bus can produce a current transient large enough to damage the generator, its prime mover, or the switchgear. The sequence in this project (Chapter 6) verifies speed, then voltage, then phase — each with its own tolerance and its own timeout — and only ever closes the breaker from the one state where every condition has just been confirmed true in that scan.

DESIGN NOTE — The incoming generator runs deliberately fast

During speed matching, the target isn't to match the bus frequency exactly — it's to run marginally faster (a small deliberate lead, implemented as the +0.05Hz bias in Chapter 6). A generator held at exactly the bus frequency has a phase angle that drifts unpredictably; one running slightly fast has a phase angle that advances steadily and predictably toward the closing window, which is what actually makes a bounded-time closing decision possible.

5.3 Blackout recovery: starting from zero, not from a fault

Blackout recovery isn't "auto-sync, but urgent" — it's a different problem because there's nothing left to synchronise against. Once every generator is confirmed down and bus voltage has genuinely collapsed, the correct action for the first generator back online is a direct dead-bus close: no phase matching, because there's no second live source to match against. Attempting to run the live-sync state machine against a dead bus would simply never satisfy the phase-window condition and stall recovery for no reason.

SAFETY CRITICAL — Dead-bus close and live sync must never share a code path

This is the single most important design decision in this project. If a generator ever closes onto a bus that turns out to still be live — because the dead-bus check was wrong, or shared logic with the live-sync path introduced a subtle bug — the result is an out-of-phase closure with no supervision at all. Section 3 of the code (Chapter 6) keeps these as two structurally separate branches specifically so a bug in one can't silently borrow the other's closing permissive.

5.4 Why restoration is staged, not instant

Once the main bus is back, dumping all previously-shed load back on at once risks re-tripping a generator that's only just stabilised. Load is restored in the exact reverse of the order it was shed — last shed, first back — with a timed gap between each stage, giving the generator's governor and AVR room to settle before the next block lands.

06

PMS POWER MANAGEMENT

PLC Logic Walkthrough

This chapter walks through FB_PMS_PowerManagement section by section. The full, unbroken listing is reproduced in Appendix B.

6.1 Bus and generator status

Every other section depends on three derived values computed here first: how many generators are actually contributing to the bus, what the plant's total load is, and whether the bus is genuinely dead (not just low, but below a defined threshold).

```
RunningGenCount := 0;
IF DI_Gen1_Running AND DI_Gen1_Breaker_Closed THEN RunningGenCount := RunningGenCount + 1; END_IF
IF DI_Gen2_Running AND DI_Gen2_Breaker_Closed THEN RunningGenCount := RunningGenCount + 1; END_IF

AvailableCapacity_kW := INT_TO_REAL(RunningGenCount) * Gen_Rated_kW;
BusIsDead := AI_BusVoltage_V < DeadBus_Threshold_V;
BlackoutDetected := DI_System_Enable AND BusIsDead
                          AND (NOT DI_Gen1_Running) AND (NOT DI_Gen2_Running);
```

Listing 6.1 — Bus and generator status, computed once per scan.

6.2 Fast load shedding

An edge-detected trip, or simply exceeding the overload margin, sheds stages immediately — note there's no PID, no ramp, just direct comparisons against pre-sized blocks. This section is explicitly skipped while a blackout recovery is in progress, so it can't fight with the forced-shed state Section 6.3 sets.

```
Gen1_TripEdge := Gen1_Was_Running AND (NOT DI_Gen1_Breaker_Closed) AND (NOT BlackoutDetected);
Gen1_Was_Running := DI_Gen1_Breaker_Closed;

IF Gen1_TripEdge OR (TotalLoad_kW > AvailableCapacity_kW * (Overload_Margin_Pct / 100.0)) THEN
    DO_LoadShed_Stage1 := TRUE;
    IF TotalLoad_kW - LoadShed_Stage1_kW > AvailableCapacity_kW * (Overload_Margin_Pct / 100.0)
THEN
        DO_LoadShed_Stage2 := TRUE;
    END_IF
    IF TotalLoad_kW - LoadShed_Stage1_kW - LoadShed_Stage2_kW > AvailableCapacity_kW *
(Overload_Margin_Pct / 100.0) THEN
        DO_LoadShed_Stage3 := TRUE;
    END_IF
    Alarm_Overload := TRUE;
ELSIF TotalLoad_kW < AvailableCapacity_kW * 0.6 THEN
    DO_LoadShed_Stage1 := FALSE; DO_LoadShed_Stage2 := FALSE; DO_LoadShed_Stage3 := FALSE;
    Alarm_Overload := FALSE;
END_IF
```

Listing 6.2 — Fast, block-based load shedding.

6.3 Blackout pre-empts everything else

This is a short section doing something structurally important: it forces every sheddable load OFF the instant a blackout is confirmed, before power even returns, so nothing re-energises in an uncontrolled rush the moment the bus comes back.

```
IF BlackoutDetected AND (PMS_State <> ST_BLACKOUT_EMERGENCY)
   AND (PMS_State <> ST_BLACKOUT_BLACKSTART) AND (PMS_State <> ST_BLACKOUT_RESTORE) THEN
    PMS_State := ST_BLACKOUT_EMERGENCY;
    DO_LoadShed_Stage1 := TRUE;
    DO_LoadShed_Stage2 := TRUE;
    DO_LoadShed_Stage3 := TRUE;
    DO_Gen2_Start := FALSE;
    RestoreStageIndex := 0;
END_IF
```

Listing 6.3 — Blackout detection forces a safe, fully-shed starting point.

6.4 The live-sync state machine (condensed)

The full sequence runs through eight states; the two decision points worth seeing directly are the phase-window check and the breaker-close confirmation, since those are where a synchronising sequence actually earns its trust.

```
ST_PHASE_MATCH:
    T_SyncWindow(IN := TRUE, PT := T_SyncWindow_PT);
    // ... governor bias kept live here (Listing continues in Appendix B) ...
    IF (AI_PhaseAngle_Deg <= 0.0) AND (AI_PhaseAngle_Deg > -PhaseClose_Window_Deg) THEN
        PMS_State := ST_CLOSE_BREAKER;
        T_BreakerConfirm(IN := TRUE, PT := T_BreakerConfirm_PT);
    END_IF
    IF T_SyncWindow.Q THEN PMS_State := ST_SYNC_FAILED; END_IF

ST_CLOSE_BREAKER:
    DO_Gen2_Breaker_Close := TRUE;
    T_BreakerConfirm(IN := TRUE, PT := T_BreakerConfirm_PT);
    IF DI_Gen2_Breaker_Closed THEN
        DO_Gen2_Breaker_Close := FALSE;
        PMS_State := ST_RUNNING_PARALLEL;
    ELSIF T_BreakerConfirm.Q THEN
        DO_Gen2_Breaker_Close := FALSE;
        PMS_State := ST_SYNC_FAILED;
    END_IF
```

Listing 6.4 — The phase-window check and breaker-close confirmation.

> **DESIGN NOTE — Every path through this state machine ends in a verified state**
> Look at the two exits from ST_CLOSE_BREAKER: the breaker only ever registers as successfully closed via confirmed feedback (DI_Gen2_Breaker_Closed), never by assuming the close command worked. If confirmation doesn't arrive before T_BreakerConfirm_PT elapses, the sequence fails safe rather than proceeding on faith.

6.5 Blackout black-start: the dead-bus path

Compare this directly against Listing 6.4. There is no frequency check, no voltage check, no phase check — because BusIsDead was already confirmed in Section 6.1, closing directly is the correct action, not a shortcut.

```
ST_BLACKOUT_BLACKSTART:
    DO_Gen1_Start := TRUE;
    T_Warmup(IN := TRUE, PT := T_Warmup_PT);
    IF DI_Gen1_Running AND T_Warmup.Q AND BusIsDead THEN
        DO_Gen1_Breaker_Close := TRUE; // Dead-bus close - no sync check needed,
    END_IF // the bus is confirmed dead, not live.
    IF DI_Gen1_Breaker_Closed THEN
        DO_Gen1_Breaker_Close := FALSE;
        PMS_State := ST_BLACKOUT_RESTORE;
    END_IF
```

Listing 6.5 — Dead-bus close: structurally separate from live synchronising.

07

PMS POWER MANAGEMENT

HMI Design & the Live Synchroscope

The HMI mockup (hmi/index.html in the repository) centres on a single-line diagram — the standard way electrical engineers read a switchboard at a glance — paired with a live synchroscope, the classic rotating-needle dial real synchronising panels have used for decades. It runs a full scripted demo on a loop: rising load, auto-sync, parallel running, a full blackout, emergency generator start, black-start, and staged restoration.

Figure 7.1 — Switchboard console mid-synchronising: Gen2 breaker still open on the single-line diagram, synchroscope needle approaching the 12 o'clock closing position, governor/AVR bias bars active.

Design decisions

- The single-line diagram is the primary view, not a supporting chart — breaker colour (green closed, red/grey open), generator circle colour, and bus-line colour all update live, exactly what a real switchboard mimic panel shows.
- The synchroscope is a real instrument, not decoration — its needle position is driven directly by the same phase-angle value the PLC's AI_PhaseAngle_Deg input represents, rotating toward the 12 o'clock position as the sequence approaches the closing window.
- Blue/amber switchboard palette — a third distinct identity in the portfolio series (after Project 01's teal compliance theme and Project 02's red safety theme), chosen because blue-dominant panels are how electrical distribution equipment is conventionally

rendered, distinct from a process or safety console.

- Flexbox over CSS Grid — the three-column detail panel below the SLD was originally built with CSS Grid, which turned out to silently fail to lay out columns at all on an older rendering engine used during screenshot generation. Rebuilt in flexbox, which behaved correctly everywhere it was tested.

DESIGN NOTE — A second compatibility bug, a second real lesson

This is the second rendering-engine compatibility issue found across the portfolio series (the first was Project 01's CSS conic-gradient/ES6 failure). The pattern is consistent: features that are perfectly standard in a modern desktop browser (conic-gradient, template literals, CSS Grid) are not universally supported by the older embedded browser engines that panel PCs and screenshot/report tooling often still run. Treating that as a real defect class, not an edge case, is itself part of what this portfolio is meant to demonstrate.

08

PMS POWER MANAGEMENT

Alarm Philosophy & Fail-Safe Design

As with the other two projects, every alarm here maps to a specific expected operator response rather than a generic warning light.

Condition	Meaning	Expected Operator Response
Alarm_Overload	Load shed stage(s) active	Confirm shed loads are non-essential; investigate cause of high load
Alarm_Blackout	Blackout recovery in progress	Monitor recovery sequence; do not manually intervene unless it stalls
Alarm_SyncTimeout	Auto-sync sequence failed to complete	Investigate generator/governor/AVR fault before re-requesting sync

Fail-safe defaults, summarised

- Any generator trip while carrying significant load → load sheds within one scan, no waiting for confirmation of cause.
- Any sync stage timeout → sequence aborts to SYNC_FAILED; breaker is never closed on an unverified condition.
- Blackout confirmed → every sheddable load forced off before any recovery action begins.
- Sync retry requires the operator to drop and re-raise the request — no silent, unattended retry loop against a fault that hasn't been investigated.

PMS POWER MANAGEMENT

09 Testing, Commissioning & FAT Procedures

The function block was validated against twelve functional test cases spanning normal operation, the full auto-sync sequence (including two deliberate failure paths), and the complete blackout-to-recovery chain. The full procedure is in docs/Testing_Procedures.md; the matrix is reproduced below.

#	Test Case	Expected Result
1	Normal single-generator operation	Status NORMAL; no shed stages active
2	Fast load shed on unexpected trip	Stages shed within one scan; Alarm_Overload raised
3	Load-dependent auto-start	Gen2 start command energises above 85% single-gen load
4	Full auto-sync sequence	Reaches RUNNING_PARALLEL; breaker confirmed closed
5	Sync timeout - frequency never matches	SYNC_FAILED; breaker never commanded closed
6	Sync retry requires a fresh request	No silent auto-retry after a failure
7	Breaker close confirmation failure	SYNC_FAILED; close command drops
8	Full blackout detection	BLACKOUT_EMERGENCY within one scan
9	Emergency generator auto-start	Start command energises; all shed stages forced TRUE
10	Black-start dead-bus close	Direct close — no synchronising states entered
11	Staged load restoration	Reverse-order restore, each stage timer-gated
12	Blackout recovery completes to normal	Returns to IDLE; Alarm_Blackout clears

ENGINEERING NOTE — Tests 5–7 matter as much as Test 4

A synchroniser that completes a clean sync (Test 4) but can't fail safely is worse than no automation at all. Tests 5 through 7 exist specifically to prove the sequence aborts cleanly under a frequency fault, a stalled attempt, and a breaker that fails to confirm — and that in every one of those cases, the breaker-close command is never left asserted against an unverified condition.

9.1 Commissioning sequence (SAT-style, for reference)

Step	Activity	Exit Criteria
1	Cold loop checks	Every I/O point traced; breaker aux. contacts verified against actual position

Step	Activity	Exit Criteria
2	Governor/AVR bias calibration	Bias signal range matched to the actual generator's droop characteristic
3	Load shed contactor test	Each stage group trips and restores correctly, independent of the others
4	Live sync proving (no load)	Full sequence run and breaker closed with both gensets lightly loaded
5	Fast shed proving	Duty generator deliberately tripped under real load; shed timing measured against the class overload trip curve
6	Blackout drill	Full plant blackout simulated; emergency gen, black-start, and restoration timed end-to-end
7	Sign-off	Class surveyor and shipowner's engineer countersign the FAT/SAT record

PMS POWER MANAGEMENT

10 Limitations, Real-World Deltas & Future Work

Naming the gap between a strong simulation and a certifiable Power Management System is part of the engineering, consistent with the other two projects in this series.

What a real installation would add

- Load sharing (droop) control — once generators are in parallel, this function block hands off governor/AVR control entirely; a real PMS actively balances kW and kVAR sharing between running generators on an ongoing basis.
- Reverse-power and other protection coordination — a real system coordinates this logic with generator protection relays (reverse power, overcurrent, loss of excitation) rather than treating them as independent.
- Auto-stop / de-load sequencing — stopping Gen2 once it's no longer needed is explicitly left as an operator decision in this design (Chapter 6.4) rather than modelled.
- Class society approval — DNV, ABS, or Lloyd's Register review against their specific PMS notation requirements, including a documented FMEA.

10.1 Hazard & safeguard register (HAZOP-style)

Hazard	Cause	Safeguard in This Design
Out-of-phase breaker closure	Sync logic closes before phase genuinely aligned	Phase window checked in the same scan as closure; dead-bus and live-sync paths structurally separate
Cascading blackout from single trip	Load shed too slow or wrong amount	Pre-sized blocks, no iterative calculation, edge-triggered within one scan
Uncontrolled load rush on recovery	All load restored simultaneously after blackout	Reverse-order, timer-gated staged restoration (Chapter 5.4)
Silent, repeated failed sync attempts	Automatic retry against an unresolved fault	Retry requires the operator to drop and re-raise the request

Where this project could go next

- Add basic kW droop-based load sharing once both generators are confirmed parallel.
- Model a third generator to demonstrate the state machine scaling beyond a two-set plant.
- Add a data-logged timing report per blackout event, comparing actual recovery time against the SOLAS 45-second and class-society expectations documented in Chapter 1.

0A

APPENDIX

Appendix A — I/O Quick Reference

Tag	Dir.	Type	Notes
AI_Gen1_Load_kW / AI_Gen2_Load_kW	IN	REAL x2	0-2200 kW
AI_Gen2_Freq_Hz / AI_BusFreq_Hz	IN	REAL x2	Tolerance 0.10 Hz
AI_Gen2_Voltage_V / AI_BusVoltage_V	IN	REAL x2	Tolerance 2.0%
AI_PhaseAngle_Deg	IN	REAL	Close window -8 to 0 deg
DI_Gen1/2_Running	IN	BOOL x2	Engine feedback
DI_Gen1/2_Breaker_Closed	IN	BOOL x2	Aux. contact
DI_EmergencyGen_Running	IN	BOOL	SOLAS emergency source
DI_ManualSyncRequest	IN	BOOL	Must re-raise after a failure
DI_System_Enable	IN	BOOL	Master enable
DO_Gen1_Start / DO_Gen2_Start	OUT	BOOL x2	Start commands
DO_Gen1_Breaker_Close	OUT	BOOL	Dead-bus only
DO_Gen2_Breaker_Close	OUT	BOOL	Live-sync only, pulsed
AO_Gen2_GovernorBias_Pct / AVRBias_Pct	OUT	REAL x2	-100..100%
DO_LoadShed_Stage1/2/3	OUT	BOOL x3	Reverse-order restore
DO_EmergencyGen_Start	OUT	BOOL	SOLAS 45s target
PMS_State	OUT	ENUM	12-state machine

APPENDIX

0B Appendix B — Full Structured Text Listing

Complete, unedited listing of src/PMS_PowerManagement.st.

```
(*
================================================================================
  PROJECT : Autonomous Marine Power Management System (PMS)
            Intelligent Load-Shedding & Blackout Recovery
  MODULE : FB_PMS_PowerManagement
  PLATFORM : IEC 61131-3 Structured Text (Siemens SCL / CODESYS-portable)
  AUTHOR : Sipho Lucky Sibanda
  REGULATORY BASIS (simulated against, for realism only):
    - SOLAS Ch. II-1, Reg 42/43 - Emergency source, 45-second start
    - IEC 61892-1 / IEC 60092 - Electrical installations on ships
================================================================================
*)

TYPE E_PMS_State :
(
    ST_IDLE, ST_STARTING, ST_WARMUP, ST_SPEED_MATCH, ST_VOLTAGE_MATCH,
    ST_PHASE_MATCH, ST_CLOSE_BREAKER, ST_RUNNING_PARALLEL, ST_SYNC_FAILED,
    ST_BLACKOUT_EMERGENCY, ST_BLACKOUT_BLACKSTART, ST_BLACKOUT_RESTORE
);
END_TYPE

FUNCTION_BLOCK FB_PMS_PowerManagement
VAR_INPUT
    AI_Gen1_Load_kW : REAL; AI_Gen2_Load_kW : REAL;
    AI_Gen2_Freq_Hz : REAL; AI_BusFreq_Hz : REAL;
    AI_Gen2_Voltage_V : REAL; AI_BusVoltage_V : REAL;
    AI_PhaseAngle_Deg : REAL;
    DI_Gen1_Running : BOOL; DI_Gen2_Running : BOOL;
    DI_Gen1_Breaker_Closed : BOOL; DI_Gen2_Breaker_Closed : BOOL;
    DI_EmergencyGen_Running : BOOL;
    DI_ManualSyncRequest : BOOL;
    DI_System_Enable : BOOL;
END_VAR
```

```
VAR_OUTPUT
    DO_Gen1_Start : BOOL := FALSE; DO_Gen2_Start : BOOL := FALSE;
    DO_Gen1_Breaker_Close : BOOL := FALSE; DO_Gen2_Breaker_Close : BOOL := FALSE;
    AO_Gen2_GovernorBias_Pct : REAL := 0.0; AO_Gen2_AVRBias_Pct : REAL := 0.0;
    DO_LoadShed_Stage1 : BOOL := FALSE;
    DO_LoadShed_Stage2 : BOOL := FALSE;
    DO_LoadShed_Stage3 : BOOL := FALSE;
    DO_EmergencyGen_Start : BOOL := FALSE;
    Alarm_Overload : BOOL := FALSE;
    Alarm_Blackout : BOOL := FALSE;
    Alarm_SyncTimeout : BOOL := FALSE;
    PMS_State : E_PMS_State := ST_IDLE;
    SystemStatus : STRING[24] := 'STANDBY';
END_VAR

VAR
    Gen_Rated_kW : REAL := 2000.0;
    LoadShed_Stage1_kW : REAL := 150.0; LoadShed_Stage2_kW : REAL := 300.0;
    LoadShed_Stage3_kW : REAL := 200.0;
    Overload_Margin_Pct : REAL := 90.0;
    Freq_Tolerance_Hz : REAL := 0.10; Volt_Tolerance_Pct : REAL := 2.0;
    PhaseClose_Window_Deg : REAL := 8.0; DeadBus_Threshold_V : REAL := 50.0;

    RunningGenCount : INT; TotalLoad_kW : REAL; AvailableCapacity_kW : REAL;
    BusIsDead : BOOL; BlackoutDetected : BOOL;
    TempBias : REAL; VoltErrPct : REAL;
    Gen1_Was_Running : BOOL; Gen1_TripEdge : BOOL;

    T_Start : TON; T_Start_PT : TIME := T#20S;
    T_Warmup : TON; T_Warmup_PT : TIME := T#30S;
    T_SyncWindow : TON; T_SyncWindow_PT : TIME := T#60S;
    T_BreakerConfirm : TON; T_BreakerConfirm_PT : TIME := T#500MS;
    T_RestoreStage : TON; T_RestoreStage_PT : TIME := T#15S;
    RestoreStageIndex : INT := 0;
END_VAR
```

```
// 1. BUS & GENERATOR STATUS
RunningGenCount := 0;
IF DI_Gen1_Running AND DI_Gen1_Breaker_Closed THEN RunningGenCount := RunningGenCount + 1; END_IF
IF DI_Gen2_Running AND DI_Gen2_Breaker_Closed THEN RunningGenCount := RunningGenCount + 1; END_IF

TotalLoad_kW := 0.0;
IF DI_Gen1_Breaker_Closed THEN TotalLoad_kW := TotalLoad_kW + AI_Gen1_Load_kW; END_IF
IF DI_Gen2_Breaker_Closed THEN TotalLoad_kW := TotalLoad_kW + AI_Gen2_Load_kW; END_IF

AvailableCapacity_kW := INT_TO_REAL(RunningGenCount) * Gen_Rated_kW;
BusIsDead := AI_BusVoltage_V < DeadBus_Threshold_V;
BlackoutDetected := DI_System_Enable AND BusIsDead
                            AND (NOT DI_Gen1_Running) AND (NOT DI_Gen2_Running);

// 2. FAST LOAD SHEDDING
Gen1_TripEdge := Gen1_Was_Running AND (NOT DI_Gen1_Breaker_Closed) AND (NOT BlackoutDetected);
Gen1_Was_Running := DI_Gen1_Breaker_Closed;

IF (PMS_State <> ST_BLACKOUT_EMERGENCY) AND (PMS_State <> ST_BLACKOUT_BLACKSTART)
   AND (PMS_State <> ST_BLACKOUT_RESTORE) THEN
    IF Gen1_TripEdge OR (TotalLoad_kW > AvailableCapacity_kW * (Overload_Margin_Pct / 100.0)) THEN
        DO_LoadShed_Stage1 := TRUE;
        IF TotalLoad_kW - LoadShed_Stage1_kW > AvailableCapacity_kW * (Overload_Margin_Pct / 100.0)
THEN
            DO_LoadShed_Stage2 := TRUE;
        END_IF
        IF TotalLoad_kW - LoadShed_Stage1_kW - LoadShed_Stage2_kW > AvailableCapacity_kW *
(Overload_Margin_Pct / 100.0) THEN
            DO_LoadShed_Stage3 := TRUE;
        END_IF
        Alarm_Overload := TRUE;
    ELSIF TotalLoad_kW < AvailableCapacity_kW * 0.6 THEN
        DO_LoadShed_Stage1 := FALSE; DO_LoadShed_Stage2 := FALSE; DO_LoadShed_Stage3 := FALSE;
        Alarm_Overload := FALSE;
    END_IF
END_IF
```

```
// 3. BLACKOUT PRE-EMPTS EVERYTHING ELSE
IF BlackoutDetected AND (PMS_State <> ST_BLACKOUT_EMERGENCY)
   AND (PMS_State <> ST_BLACKOUT_BLACKSTART) AND (PMS_State <> ST_BLACKOUT_RESTORE) THEN
    PMS_State := ST_BLACKOUT_EMERGENCY;
    DO_LoadShed_Stage1 := TRUE; DO_LoadShed_Stage2 := TRUE; DO_LoadShed_Stage3 := TRUE;
    DO_Gen2_Start := FALSE;
    RestoreStageIndex := 0;
END_IF

// 4. LOAD-DEPENDENT AUTO-START TRIGGER
IF DI_System_Enable AND (NOT BlackoutDetected) AND (PMS_State = ST_IDLE)
   AND ((TotalLoad_kW > Gen_Rated_kW * 0.85) OR DI_ManualSyncRequest)
   AND (NOT DI_Gen2_Breaker_Closed) THEN
    PMS_State := ST_STARTING;
END_IF

// 5. AUTO-SYNCHRONISING STATE MACHINE (Gen2 onto a LIVE bus)
CASE PMS_State OF
    ST_STARTING:
        DO_Gen2_Start := TRUE;
        T_Start(IN := TRUE, PT := T_Start_PT);
        IF DI_Gen2_Running THEN
            T_Start(IN := FALSE); T_Warmup(IN := TRUE, PT := T_Warmup_PT);
            PMS_State := ST_WARMUP;
        ELSIF T_Start.Q THEN
            PMS_State := ST_SYNC_FAILED;
        END_IF

    ST_WARMUP:
        T_Warmup(IN := TRUE, PT := T_Warmup_PT);
        IF T_Warmup.Q THEN
            T_Warmup(IN := FALSE); T_SyncWindow(IN := TRUE, PT := T_SyncWindow_PT);
            PMS_State := ST_SPEED_MATCH;
        END_IF
```

```
    ST_SPEED_MATCH:
        T_SyncWindow(IN := TRUE, PT := T_SyncWindow_PT);
        TempBias := (AI_BusFreq_Hz + 0.05 - AI_Gen2_Freq_Hz) * 50.0;
        IF TempBias > 100.0 THEN TempBias := 100.0; END_IF
        IF TempBias < -100.0 THEN TempBias := -100.0; END_IF
        AO_Gen2_GovernorBias_Pct := TempBias;
        IF ABS(AI_BusFreq_Hz - AI_Gen2_Freq_Hz) <= Freq_Tolerance_Hz THEN
            PMS_State := ST_VOLTAGE_MATCH;
        END_IF
        IF T_SyncWindow.Q THEN PMS_State := ST_SYNC_FAILED; END_IF

    ST_VOLTAGE_MATCH:
        T_SyncWindow(IN := TRUE, PT := T_SyncWindow_PT);
        VoltErrPct := ((AI_BusVoltage_V - AI_Gen2_Voltage_V) / AI_BusVoltage_V) * 100.0;
        TempBias := VoltErrPct * 20.0;
        IF TempBias > 100.0 THEN TempBias := 100.0; END_IF
        IF TempBias < -100.0 THEN TempBias := -100.0; END_IF
        AO_Gen2_AVRBias_Pct := TempBias;
        IF ABS(VoltErrPct) <= Volt_Tolerance_Pct THEN
            PMS_State := ST_PHASE_MATCH;
        END_IF
        IF T_SyncWindow.Q THEN PMS_State := ST_SYNC_FAILED; END_IF

    ST_PHASE_MATCH:
        T_SyncWindow(IN := TRUE, PT := T_SyncWindow_PT);
        TempBias := (AI_BusFreq_Hz + 0.05 - AI_Gen2_Freq_Hz) * 50.0;
        IF TempBias > 100.0 THEN TempBias := 100.0; END_IF
        IF TempBias < -100.0 THEN TempBias := -100.0; END_IF
        AO_Gen2_GovernorBias_Pct := TempBias;
        IF (AI_PhaseAngle_Deg <= 0.0) AND (AI_PhaseAngle_Deg > -PhaseClose_Window_Deg) THEN
            PMS_State := ST_CLOSE_BREAKER;
            T_BreakerConfirm(IN := TRUE, PT := T_BreakerConfirm_PT);
        END_IF
        IF T_SyncWindow.Q THEN PMS_State := ST_SYNC_FAILED; END_IF

    ST_CLOSE_BREAKER:
        DO_Gen2_Breaker_Close := TRUE;
        T_BreakerConfirm(IN := TRUE, PT := T_BreakerConfirm_PT);
        IF DI_Gen2_Breaker_Closed THEN
            DO_Gen2_Breaker_Close := FALSE; T_BreakerConfirm(IN := FALSE); T_SyncWindow(IN :=
FALSE);
            AO_Gen2_GovernorBias_Pct := 0.0; AO_Gen2_AVRBias_Pct := 0.0;
            PMS_State := ST_RUNNING_PARALLEL;
        ELSIF T_BreakerConfirm.Q THEN
            DO_Gen2_Breaker_Close := FALSE; PMS_State := ST_SYNC_FAILED;
        END_IF

    ST_RUNNING_PARALLEL:
        IF NOT DI_Gen2_Breaker_Closed THEN PMS_State := ST_IDLE; END_IF

    ST_SYNC_FAILED:
        DO_Gen2_Start := FALSE; Alarm_SyncTimeout := TRUE;
        AO_Gen2_GovernorBias_Pct := 0.0; AO_Gen2_AVRBias_Pct := 0.0;
        IF NOT DI_ManualSyncRequest THEN
            Alarm_SyncTimeout := FALSE; PMS_State := ST_IDLE;
        END_IF
```

```
    ST_BLACKOUT_EMERGENCY:
        DO_EmergencyGen_Start := TRUE;
        Alarm_Blackout := TRUE;
        IF DI_EmergencyGen_Running THEN
            T_Warmup(IN := TRUE, PT := T_Warmup_PT);
            PMS_State := ST_BLACKOUT_BLACKSTART;
        END_IF

    ST_BLACKOUT_BLACKSTART:
        DO_Gen1_Start := TRUE;
        T_Warmup(IN := TRUE, PT := T_Warmup_PT);
        IF DI_Gen1_Running AND T_Warmup.Q AND BusIsDead THEN
            DO_Gen1_Breaker_Close := TRUE;
        END_IF
        IF DI_Gen1_Breaker_Closed THEN
            DO_Gen1_Breaker_Close := FALSE; T_Warmup(IN := FALSE);
            T_RestoreStage(IN := TRUE, PT := T_RestoreStage_PT);
            PMS_State := ST_BLACKOUT_RESTORE;
        END_IF

    ST_BLACKOUT_RESTORE:
        T_RestoreStage(IN := TRUE, PT := T_RestoreStage_PT);
        CASE RestoreStageIndex OF
            0: IF T_RestoreStage.Q THEN
                   DO_LoadShed_Stage3 := FALSE; T_RestoreStage(IN := FALSE); RestoreStageIndex :=
1;
               END_IF
            1: IF T_RestoreStage.Q THEN
                   DO_LoadShed_Stage2 := FALSE; T_RestoreStage(IN := FALSE); RestoreStageIndex :=
2;
               END_IF
            2: IF T_RestoreStage.Q THEN
                   DO_LoadShed_Stage1 := FALSE; T_RestoreStage(IN := FALSE); RestoreStageIndex :=
3;
               END_IF
            3: Alarm_Blackout := FALSE; RestoreStageIndex := 0; PMS_State := ST_IDLE;
        END_CASE

    ELSE
        ; // ST_IDLE
END_CASE

// 6. STATUS TEXT
IF NOT DI_System_Enable THEN SystemStatus := 'STANDBY';
ELSIF BlackoutDetected OR (PMS_State = ST_BLACKOUT_EMERGENCY)
      OR (PMS_State = ST_BLACKOUT_BLACKSTART) OR (PMS_State = ST_BLACKOUT_RESTORE) THEN
    SystemStatus := 'BLACKOUT RECOVERY';
ELSIF PMS_State = ST_RUNNING_PARALLEL THEN SystemStatus := 'GENS PARALLEL';
ELSIF PMS_State = ST_SYNC_FAILED THEN SystemStatus := 'SYNC FAILED';
ELSIF (PMS_State <> ST_IDLE) THEN SystemStatus := 'SYNCHRONISING';
ELSIF Alarm_Overload THEN SystemStatus := 'LOAD SHED ACTIVE';
ELSE SystemStatus := 'NORMAL - GEN1 DUTY';
END_IF

END_FUNCTION_BLOCK
```

Listing B.1 — Complete FB_PMS_PowerManagement source.

0C

APPENDIX

Appendix C — Glossary

Term	Meaning
PMS	Power Management System — supervises generators, load sharing, and shutdown/start sequencing
SOLAS	Safety of Life at Sea — the IMO convention setting the 45-second emergency power requirement
Blackout	Total loss of electrical power across the main switchboard
Black start	Starting and energising a generator/bus with no external power source available
Dead-bus close	Closing a breaker onto a confirmed de-energised bus — no synchronising required
Synchronising	Matching an incoming generator's speed, voltage, and phase to a live bus before closing its breaker
Synchroscope	An instrument (physical or, here, simulated) showing phase alignment between two AC sources
Droop	A governor/AVR control method that shares load proportionally between parallel generators
Load shedding	Automatically disconnecting non-essential loads to protect remaining generation capacity
AVR	Automatic Voltage Regulator — controls a generator's terminal voltage
Governor	Controls a generator's prime mover speed, and therefore frequency
HAZOP	Hazard and Operability study — a structured method for identifying process risks

CLOSING

About the Author

Sipho Lucky Sibanda is an automation and controls engineer building a multi-disciplinary portfolio spanning marine systems, industrial automation, biotech, and applied AI/PLC integration. This manual is the third in the Marine Automation Portfolio series, following the same documentation standard as Projects 01 and 02: full PLC logic, I/O documentation, a live HMI, functional test procedures, and an honest account of what separates a strong simulation from a certifiable production system.

Repository: marine-pms-loadshare

End of document.